Vectored Interrupt Manager (VIM) Module

This chapter describes the behavior of the vectored interrupt manager (VIM) module of the device family.

Topic **Page**

15.1 Overview 539
15.2 Device Level Interrupt Management 539
15.3 Interrupt Handling Inside VIM 542
15.4 Interrupt Vector Table (VIM RAM) 546
15.5 VIM Wakeup Interrupt 549
15.6 Capture Event Sources 550
15.7 Examples 550
15.8 VIM Registers 553

15.1 Overview

The vectored interrupt manager (VIM) provides hardware assistance for prioritizing and controlling the many interrupt sources present on a device. Interrupts are caused by events outside of the normal flow of program execution. Normally, these events require a timely response from the central processing unit (CPU); therefore, when an interrupt occurs, the CPU switches execution from the normal program flow to an interrupt service routine (ISR).

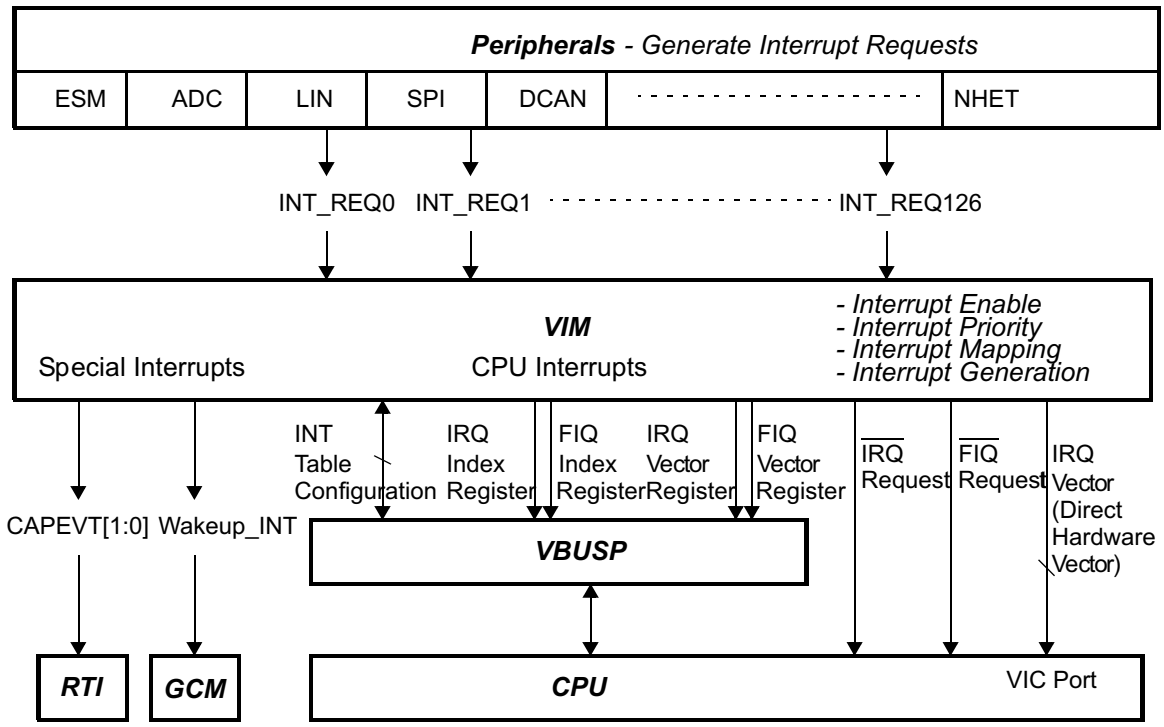
The VIM module has the following features:

- Supports 128 interrupt channels, in both register vectored interrupt and hardware vectored interrupt mode.
  - Provides IRQ vector directly to the CPU VIC port
  - Provides FIQ/IRQ vector through registers
  - Provides programmable priority and enable for interrupt request lines
- Provides a direct hardware dispatch mechanism for fastest IRQ dispatch.
- Provides two software dispatch mechanisms for backward compatibility with earlier generation of TI processors.
  - Index interrupt
  - Register vectored interrupt
- Parity protected vector interrupt table against soft errors.

15.2 Device Level Interrupt Management

A block diagram of device level interrupt handling is shown in Figure 15-1. When an event occurs within a peripheral, the peripheral makes an interrupt request to the VIM. Then, VIM prioritizes the requests from peripherals and provides the address of the highest interrupt service routine (ISR) to the CPU. Finally, CPU starts executing the ISR instructions from that address in the ISR. Section 15.2.1 through Section 15.2.3 provide additional details about these three steps.

Figure 15-1. Device Level Interrupt Block Diagram



15.2.1 Interrupt Generation at the Peripheral

Interrupt generation begins when an event occurs within a peripheral module. Some examples of interrupt-capable events are expiration of a counter within a timer module, receipt of a character in a communications module, and completion of a conversion in an analog-to-digital converter (ADC) module. Some device peripherals are capable of requesting interrupts on more than one interrupt request line.

Interrupts are not always generated when an event occurs; the peripheral must make an interrupt request to the VIM based on the event occurrence. Typically, the peripheral contains:

- An interrupt flag bit for each event to signify the event occurrence.
- An interrupt enable bit to control whether the event occurrence causes an interrupt request to the VIM.

15.2.2 Interrupt Handling at the CPU

The ARM CPU provides two vectors for interrupt requests—fast interrupt requests (FIQs) and normal interrupt requests (IRQs). FIQs are higher priority than IRQs, and FIQ interrupts may interrupt IRQ interrupts.

NOTE: The FIQ implemented in Cortex-R4F is Non-Maskable Fast Interrupts (NMFI). Once FIQ is enabled (by clearing F bit in CPSR), it can NOT be disabled by setting F bit in CPSR. Only a reset or an FIQ will be able to set the F bit in CPSR. By hardware, Non Maskable FIQ are not reentrant.

After reset (power reset or warm reset), both FIQ and IRQ are disabled. The CPU may enable these interrupt request channels individually within the CPSR (Current Program Status Register); CPSR bits 6 and 7 must be cleared to enable the FIQ (bit 6) and IRQ (bit 7) interrupt requests at the CPU. CPSR is writable in privilege mode only. Example 15-2 shows how to enable the IRQ and FIQ through CPSR.

When the CPU receives an interrupt request, the CPSR mode field changes to either FIQ or IRQ mode. When an IRQ interrupt is received, the CPU disables other IRQ interrupts by setting CPSR bit 7. When an FIQ interrupt is received, the CPU disables both IRQ and FIQ interrupts by setting CPSR bits 6 and 7.

A write of 1 to CPSR bit 7 disables the IRQ from CPU. However, a write of 1 to CPSR bit 6 leaves it unchanged. Example 15-2 also shows how to disable the IRQ through CPSR.

15.2.3 Software Interrupt Handling Options

The device supports three different possibilities for software to handle interrupts

1. Index interrupts mode (compatible with TMS470R1x legacy code)

   After the interrupt is received by the CPU, the CPU branches to 0x18 (IRQ) or 0x1C (FIQ) to execute the main ISR. The main ISR routine reads the offset register (IRQINDEX, FIQINDEX) to determine the source of the interrupt.

   This mode is compatible with the TMS470R1x (CIM) module and provides the same interrupt registers.

   This mode could be used if legacy code needs to be reused, porting it from the TMS470R1x family. However, imported software will not benefit from the VIM improvements.

   To port legacy software, the interrupt vector at 0x18 (IRQ) or 0x1C (FIQ) only needs to be a branch statement to a software interrupt table. The software interrupt table reads the pending interrupt from a vector offset register (FIQINDEX[7:0] for FIQ interrupts and IRQINDEX[7:0] for IRQ interrupts). All pending interrupts can be viewed in the INTREQ register. Example 15-4 shows how to respond to FIQ with short latency in this mode.

2. Register vectored interrupts (automatically provide vector address to application)

   Before enabling interrupts, the application software also has to initiate the interrupt vector table (VIM RAM).

   Once the VIM receives an interrupt, it loads the address of ISR from interrupt vector table, and store it into the interrupt vector register (IRQVECREG for IRQ interrupt, FIQVECREG for FIQ interrupt).

   After the interrupt is received by the CPU, the CPU executes the instruction placed at 0x18 or 0x1C (IRQ or FIQ vector) to load the address of ISR (interrupt vector) from the interrupt vector register. Example 15-3 illustrates the configuration for the exception vectors using this mode.

3. Hardware vectored interrupts (automatically dispatch to ISR, IRQ only)

   Before enabling interrupts, the application software must initiate the interrupt vector table (VIM RAM) pointing to the ISR for each interrupt channel.

   After the interrupt (IRQ) is received by the CPU, CPU reads the address of ISR directly from the interface with VIM (VIC port) instead of branching to 0x18. The CPU will branch directly to the ISR.

   The hardware vectored interrupt behavior must be explicitly enabled by setting the vector enable (VE) bit in the CP15 R1 register. This bit resets to 0, so that the default state after reset is backward compatible to earlier ARM CPU. Example 15-1 shows how to enable the hardware vectored interrupt.

   NOTE: This mode is NOT available for FIQ.

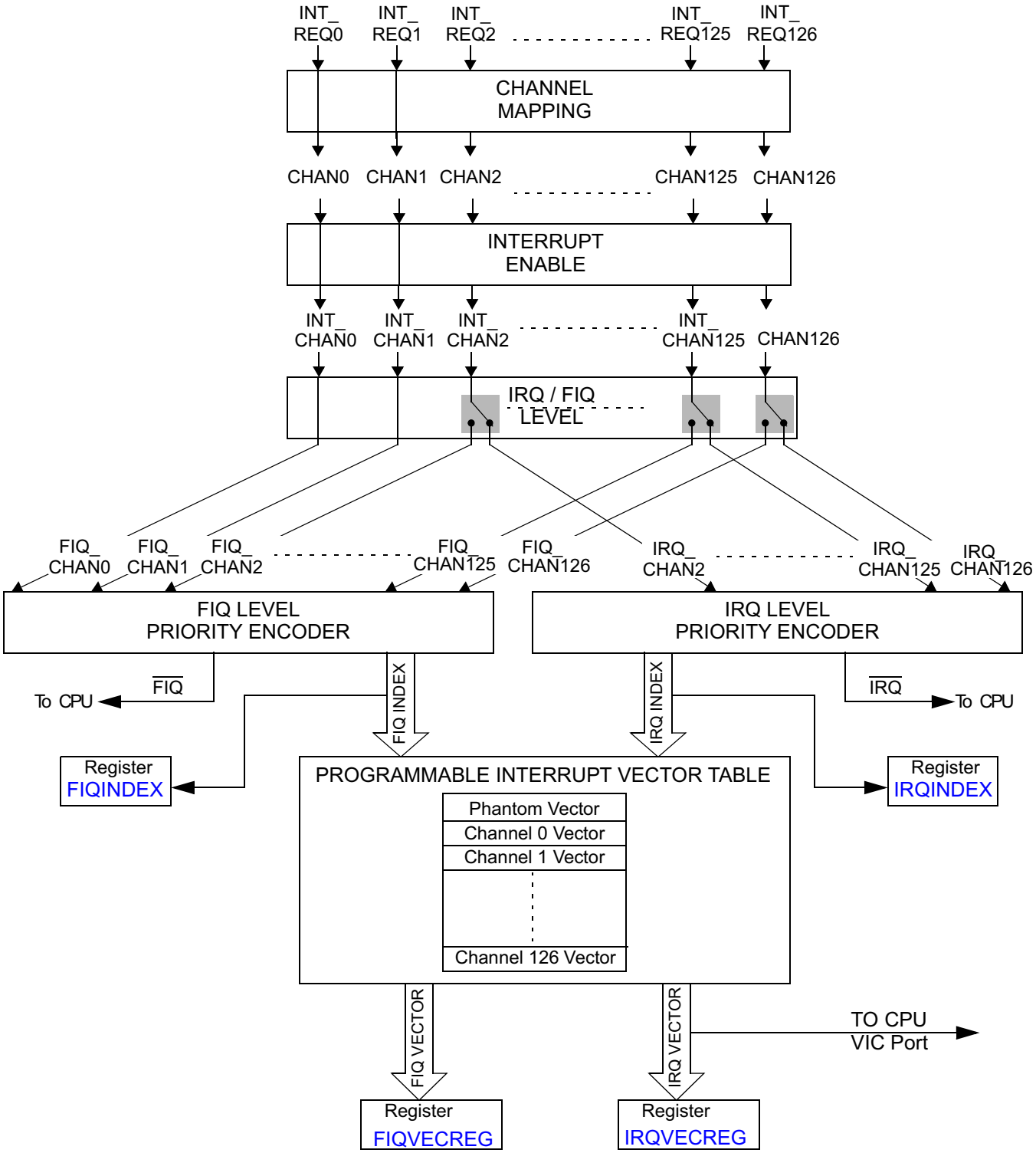
4. Software-Based Priority Decoding Scheme

   If the application uses a software-based interrupt priority decoding scheme instead of the hardware vector capabilities, then there is an additional step which was not required on earlier devices. This version of the VIM will hold an interrupt request generated by a peripheral. When the software clears the interrupt condition in the source module (for example, RTI, GIO, and so on), then it must also perform an additional clear of the interrupt request in the VIM. This can be done by reading the IRQVECREG register (Section 15.8.12) or FIQVECREG register (Section 15.8.13), or by writing a 1 to the INTREQ(i) bit (Section 15.8.7) in the VIM. This is not necessary if any of the three previous methods are used as the interrupt request bit in the VIM will be automatically cleared when the vector is read.

15.3 Interrupt Handling Inside VIM

A block diagram of the interrupt handling inside VIM is shown in Figure 15-2

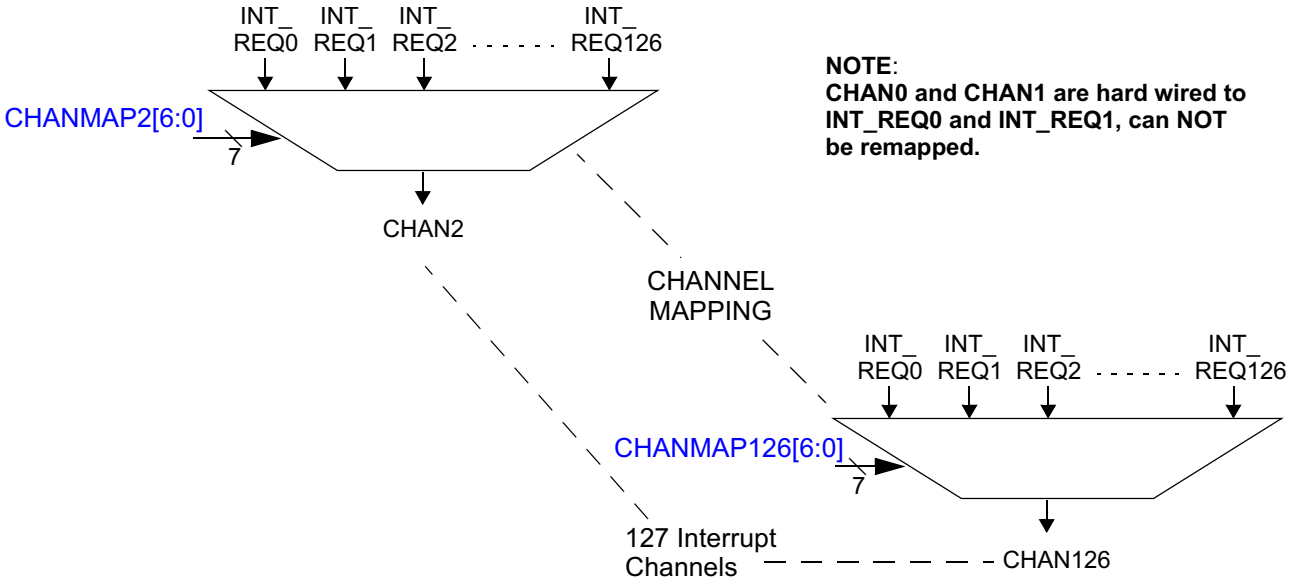
Figure 15-2. VIM Interrupt Handling Block Diagram



15.3.1 VIM Interrupt Channel Mapping

The VIM supports 128 interrupt channels (including phantom interrupt). A block diagram of the VIM interrupt requests arrangement from peripheral modules to the interrupt channels is provided in Figure 15-3. Each interrupt channel (CHANx) has a corresponding mapping register bit field (CHANMAPx[6:0]). This mapping register determines which interrupt channel it maps each VIM interrupt request. With this scheme, the same request can be mapped to multiple channels. A lower numbered channel in each FIQ and IRQ has higher priority. The programmability of the VIM allows software to control the interrupt priority.

Figure 15-3. VIM Channel Mapping



NOTE: CHAN127

CHAN127 has no dedicated interrupt vector table entry. Therefore, CHAN127 shall NOT be remapped to other INT_REQ (INT_REQ127 is reserved at device level).

In the reset state, the VIM maps all of the interrupt requests in the system to their respective interrupt channels. Figure 15-4 shows the default state following the reset.

Figure 15-5 shows the VIM INT2 is remapped to both Channel 2 and 4, and INT3 is mapped to channel 3.

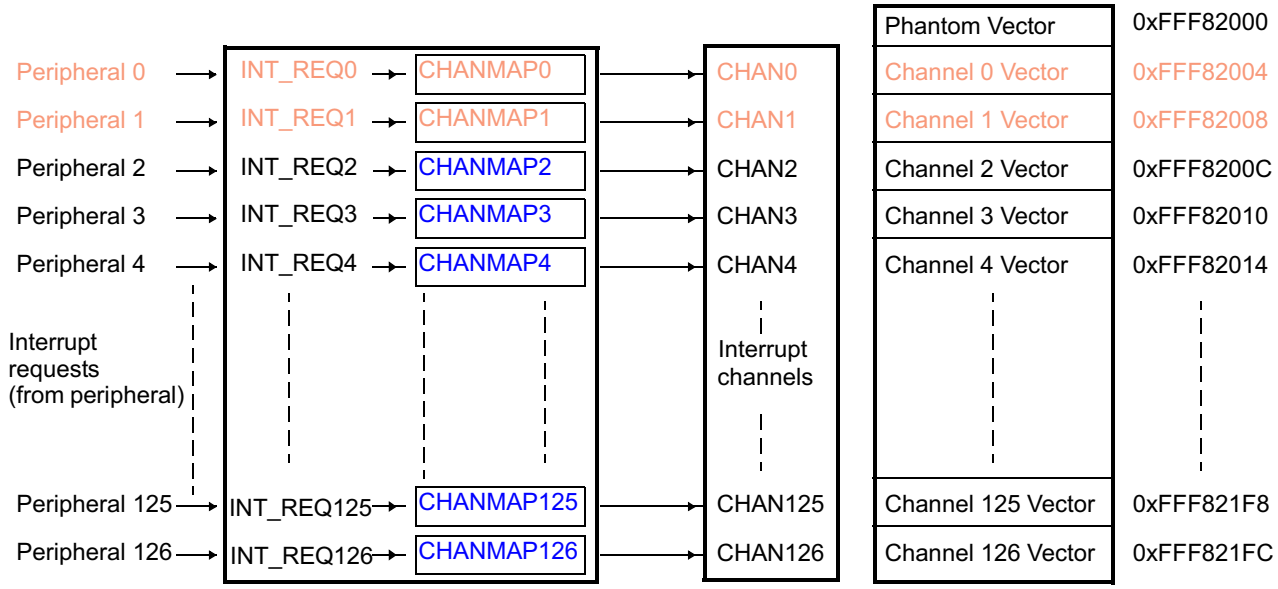
NOTE: By mapping INT2 to channel 2 and channel 4, and mapping INT3 to channel 3, it is possible for the software to change the priority dynamically by changing the ENABLE register (REQENASET and REQENACLR). When channel 2 is enabled, the priority is:

1. INT0
2. INT1
3. INT2
4. INT3

Disabling channel 2, the priority becomes:

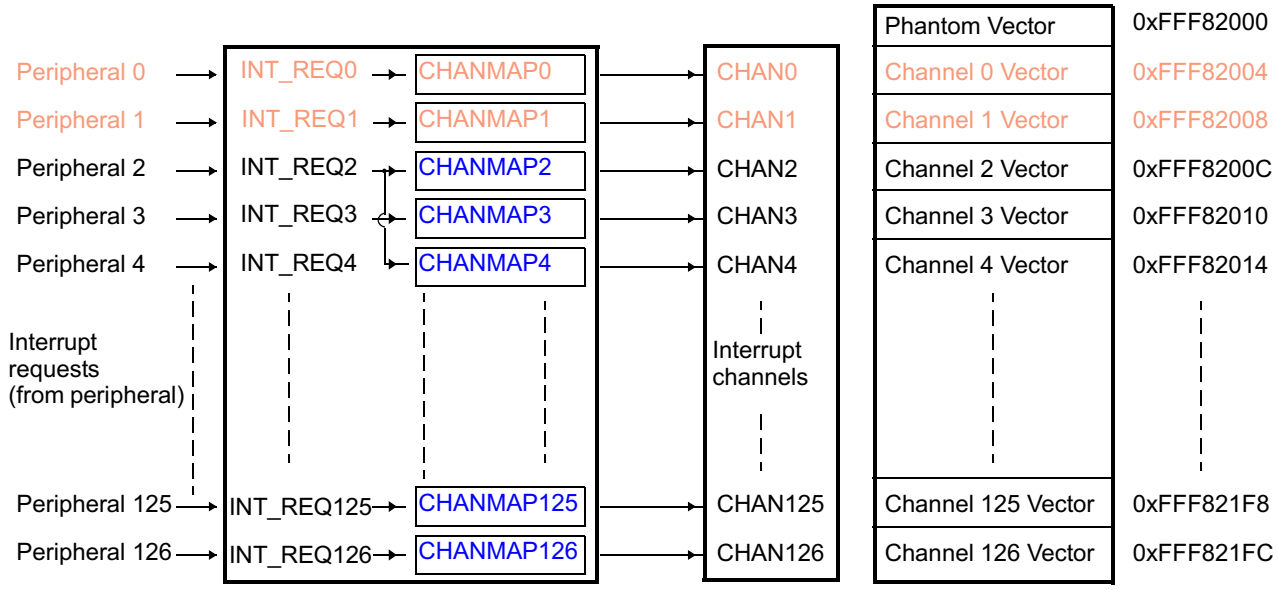
1. INT0
2. INT1
3. INT3
4. INT2

Figure 15-4. VIM in Default State



NOTE: CHAN0 and CHAN1 are hardwired to INT_REQ0 and INT_REQ1, so they cannot be remapped.

Figure 15-5. VIM in a Programmed State



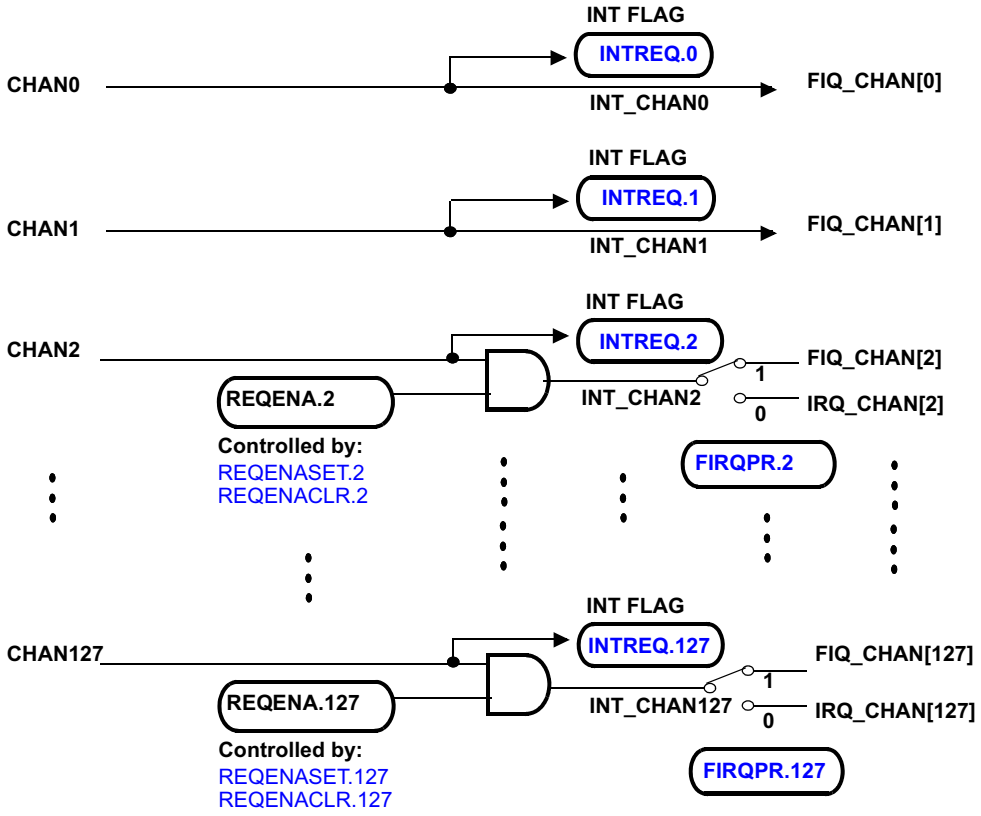
NOTE: CHAN0 and CHAN1 are hard wired to INT_REQ0 and INT_REQ1, so they cannot be remapped.

15.3.2 VIM Input Channel Management

As shown in Figure 15-6, the VIM enables channels on a channel-by-channel basis (in the REQENASET and REQENACLR registers); unused channels may be masked to prevent spurious interrupts.

NOTE: The interrupt ENABLE register does not affect the value of INTREQ.

Figure 15-6. Interrupt Channel Management



By default, interrupt CHAN0 is mapped to ESM (Error Signal Module) high level interrupt and CHAN1 is reserved for other NMI. For safety reasons, these two channels are mapped to FIQ only and can **NOT** be disabled through ENABLE registers.

NOTE: NMI Channel

Channel 0 and channel 1 are not maskable by the REQENASET and REQENACLR bit and both channel are routed exclusively to FIQ/NMI request line (FIRQPR0 and FIRQPR1 have no effect).

The VIM prioritizes the received interrupts based upon a programmed prioritization scheme. The VIM can send two interrupt requests to the CPU simultaneously—one IRQ and one FIQ. If both interrupt types are enabled at the CPU level, then the FIQ has greater priority and is handled first. Each interrupt channel, except channel 0 and 1, can be assigned to send either an FIQ or IRQ request to the CPU (in the FIRQPR register).

The VIM provides a default prioritization scheme, which sends the lowest numbered active channel (in each FIQ and IRQ classes) to the CPU. Within the FIQ and IRQ classes of interrupts, the lowest channel has the highest priority interrupt. The channel number is programmable through register CHANMAPx.

After the VIM has generated the vector corresponding to the highest active IRQ, it updates the FIQINDEX or the IRQINDEX register, depending on the class of interrupt. Then, it accesses the interrupt vector table using the vector value to fetch the address of the corresponding ISR. If the request is an FIQ class interrupt, the address read from the interrupt vector table, is written to the FIQVECREG register. If the request is an IRQ class interrupt, the address is written to the IRQVECREG register and put on the VIC port of the CPU (in case of hardware vectored interrupt is enabled).

All of the interrupt registers are updated when a new high priority interrupt line becomes active.

15.4 Interrupt Vector Table (VIM RAM)

Interrupt vector table stores the address of ISRs. During register vectored interrupt and hardware vectored interrupt, VIM accesses the interrupt vector table using the vector value to fetch the address of the corresponding ISR.

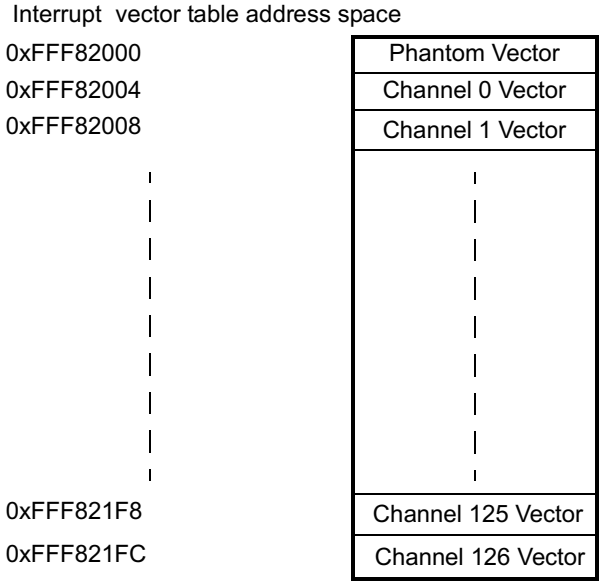
For safety reasons, the interrupt vector table has protection by parity to indicate corruption due to soft errors. The parity scheme is implemented as a continuous background check based on memory access. Section 15.4.1 through Section 15.4.4 describe how parity works in the interrupt vector table.

NOTE: Writes to the interrupt vector table parity flag register (PARFLG) and the interrupt vector table parity control register (PARCTL) are in privilege mode only.

15.4.1 Interrupt Vector Table Operation

The interrupt vector table is organized in 128 words of 32 bits. 32-bit, 16-bit, and 8-bit accesses are supported (when parity is disabled). Figure 15-7 shows the interrupt memory mapping. The table base address is FFF8 2000h.

Figure 15-7. VIM Interrupt Address Memory Map



NOTE: The interrupt vector table only has 128 entries, one phantom vector and 127 interrupt channels. Channel 127 does not have a dedicated vector and shall not be used.

There is one bit of parity per 32-bit ISR address. When a write is performed into the interrupt vector table, the parity is calculated for the 32-bit word and a parity bit is written into the corresponding parity region of interrupt vector table.

NOTE: Only 32-bit write/read access are allowed on interrupt vector table if parity is required. Non 32-bit access might result in parity errors.

When a read occurs from the CPU or VIM, the VIM calculates the parity from the data coming from the interrupt vector table and compares it to the parity stored in the table. The access of the data and the parity is performed in the same clock cycle.

If the parity bit does not match the calculated parity, a parity error is generated and the VIM stores the address of the error in the ADDERR register. The parity flag error (PARFLG) is set.

NOTE: The PARFLG register is only for bypassing the interrupt vector table in case of a parity fault. It should be used only to maintain the interrupt vector table bypassed. The checking of the parity fault should be done in the error signaling module (ESM) module where all parity errors are flagged.

Since the interrupt vector table may have an error, the FBPARERR register will provide to the VIC port, IRQVECREG and FIQVECREG, a fall-back address to an ISR that can restore the interrupt vector table content. The FBPARERR register should be set before initializing the interrupt in the interrupt vector table, to avoid branching to an unpredictable location.

The normal operation is restored when the PARFLG is cleared by the CPU. It is recommended to restore the content of the VIM before clearing the PARFLG.

The parity error signal is forwarded to the ESM.

15.4.2 Enabling and Controlling the VIM Parity

The polarity of VIM parity is controlled by the DEVCR1 register in the system module (address FFFF FFDCh). The parity enable is controlled by the PARCTL register. After reset, the parity is disabled.

Parity checking can be enabled by writing 0xA (1010b) in the PARENA[3:0] bit field of the PARCTL register. The default polarity is odd. The polarity can be changed to even by writing 5 (0101b) in the DEVPARSEL[3:0] bit field of the DEVCR1 register.

15.4.3 Interrupt Vector Table Initialization

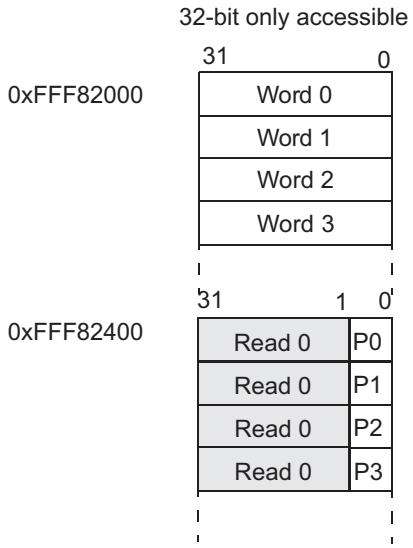
After reset, the interrupt vector table content including the parity bits is not initialized. Therefore, the CPU needs to initialize all of the interrupt addresses into the table, before enabling the corresponding interrupt channel. If parity is required, this initialization should be done after the parity functionality is enabled. In this way, the corresponding parity bit will be automatically updated. This initialization is only required when vectored interrupts are used, index interrupt management does not need the table to be initialized.

15.4.4 Interrupt Vector Table Parity Testing

To test the parity checking mechanism, the parity RAM allows manual insertion of faults. This option is implemented by the test bit in the PARCTL register. Once the bit is set, the parity bits are mapped to 0xFFF82400. After that, user can force faults into the parity bits. Finally, the parity error can be triggered by reading interrupt vector table (not parity bit) from VIM or CPU.

The interrupt vector table parity can also be verified by inserting faults into interrupt vector table. Once the VIM parity is disabled in system module, user can modify interrupt vector table without impacting the parity bit. After user re-enable interrupt vector table parity, the parity error can be triggered by reading interrupt vector table from VIM or CPU.

Figure 15-8. Parity Bit Mapping

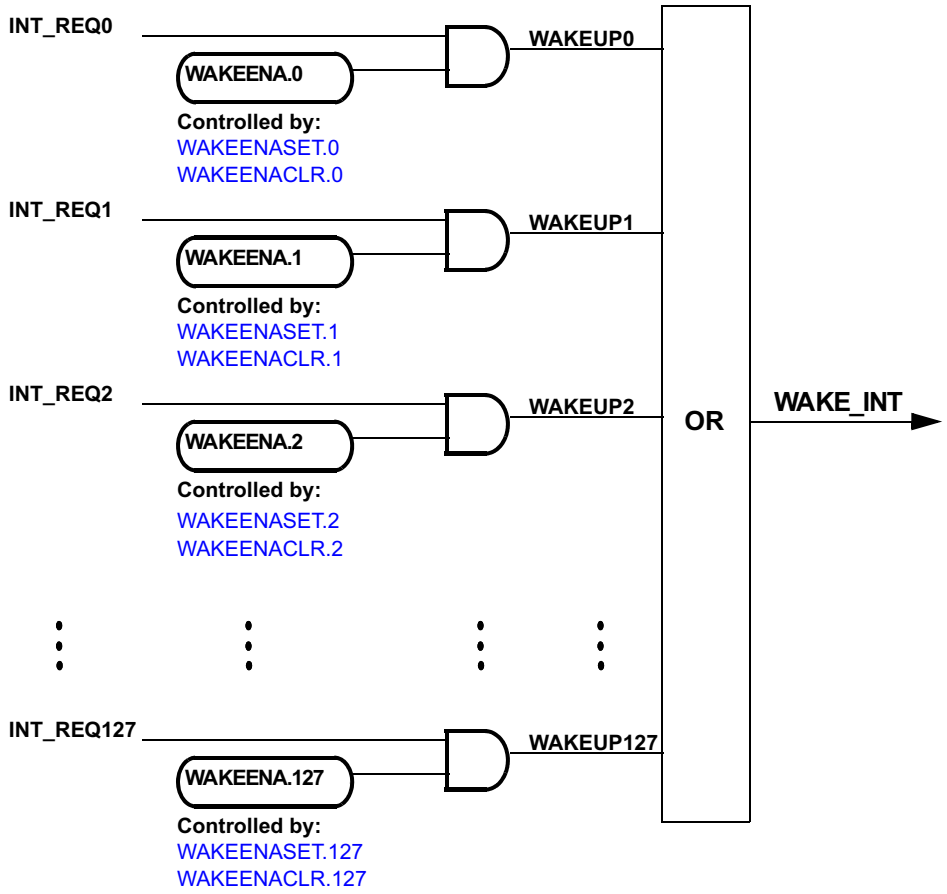


15.5 VIM Wakeup Interrupt

The wakeup interrupts are used to come out of low power mode (LPM). Any interrupt requests can be used to wake up the device. After reset, all interrupt requests are set to wake up from LPM. However, the VIM can mask unwanted interrupt lines for wake-up by using the WAKEENASET and WAKEENACLR register. The value in REQENASET / REQENACLR does NOT impact the wakeup interrupt.

As shown in Figure 15-9, the WAKEENASET and WAKEENACLR registers will enable/disable an interrupt for wake-up from low-power mode. All wake-up interrupts are “ORed” into a single signal WAKE_INT connected to the Global Clock Module.

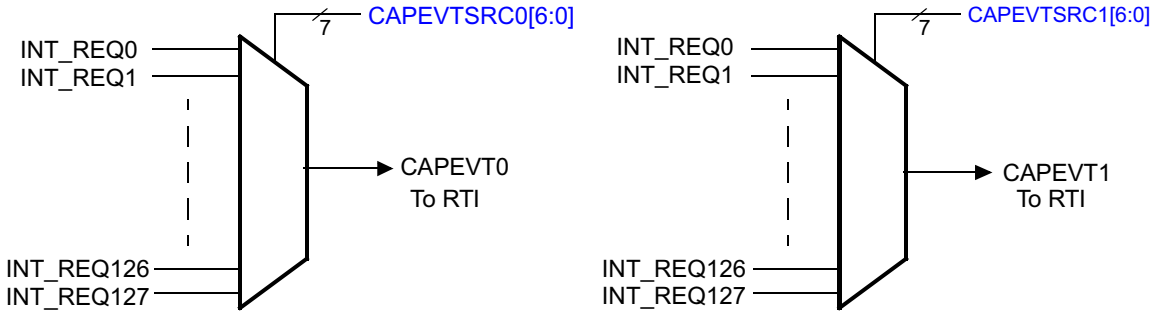
Figure 15-9. Detail of the IRQ Input



15.6 Capture Event Sources

The VIM can select any of the 128 interrupt requests to generate up to two capture events for the real-time interrupt (RTI) module (see Figure 15-10). The value in REQENASET / REQENACLR does NOT impact the capture event. Two registers (Section 15.8.14) are available, one for each capture event source.

Figure 15-10. Capture Event Sources



15.7 Examples

The following sections provide examples about the operation of the VIM.

15.7.1 Examples - Configure CPU to Receive Interrupts

Example 15-1 shows how to set the vector enable (VE) bit in the CP15 R1 register to enable the hardware vector interrupt. Example 15-2 shows how to enable/disable the IRQ and FIQ through CPSR. As a convention, the program who calls these subroutines shall preserve register R1 if needed. Example 15-2 can ONLY run in privileged mode. However, in USER mode, the application software can force the program into software interrupt by instruction 'SWI'. Then, in the software interrupt service routine, user can write register SPSR, which is the copy of CPSR in this exception mode.

Example 15-1. Enable Hardware Vector Interrupt (IRQ Only)

```
_HW_Vec_Init
    MRC p15 ,#0 ,R1 ,c1 ,c0 ,#0
    ORR R1 ,R1 ,#0x01000000       ; Mask 0-31 bits except bit 24 in Sys
                                  ; Ctrl Reg of CORTEX-R4
    MCR p15 ,#0 ,R1 ,c1 ,c0 ,#0   ; Enable bit 24
    MOV PC, LR
```

Example 15-2. Enable/Disable IRQ/FIQ through CPSR

```
FIQENABLE .equ 0x40
IRQENABLE .equ 0x80
......
_Enable_Fiq
    MRS R1, CPSR
    BIC R1, R1, #FIQENABLE
    MSR CPSR, R1
    MOV PC, LR
......
_Disable_Irq
    MRS R1, CPSR
    ORR R1, R1, #IRQENABLE
    MSR CPSR, R1
    MOV PC, LR
......
_Enable_Irq
    MRS R1, CPSR
    BIC R1, R1, #IRQENABLE
    MSR CPSR, R1
    MOV PC, LR
```

15.7.2 Examples - Register Vector Interrupt and Index Interrupt Handling

Example 15-3 illustrates the configuration for the exception vectors in Register Vector Interrupt handling. After the interrupt is received by the CPU, the CPU branches to 0x18 (IRQ) or 0x1C (FIQ). The instruction placed here should be *LDR PC, [PC,#-0x1B0]*. The pending ISR address is written into the corresponding vector register (IRQVECREG for IRQ, FIQVECREG for FIQ). The CPU reads the content of the register and branches to the ISR.

Example 15-3. Exception Vector Configuration for VIM Vector

```
          .sect ".intvecs"
00000000h b _RESET             ; RESET interrupt
00000004h b _UNDEF_INST_INT    ; UNDEFINED INSTRUCTION interrupt
00000008h b _SW_INT            ; SOFTWARE interrupt
0000000Ch b _ABORT_PREF_INT    ; ABORT (PREFETCH) interrupt
00000010h b _ABORT_DATA_INT    ; ABORT (DATA) interrupt
00000014h b #-8                ; Reserved
00000018h ldr pc,[pc,#-0x1B0]  ; IRQ interrupt
0000001Ch ldr pc,[pc,#-0x1B0]  ; FIQ interrupt
```

NOTE: Program Counter (PC) always pointers two instructions beyond the current executed instruction. In this case, PC equals to '*0x18 or 0x1C + 0x08*'. The *LDR* instruction load the memory at '*PC - 0x1B0*', which is '*0x18 or 0x1C + 0x08 - 0x1B0* = 0xFFFFFE70 or 0xFFFFFE74'. These are the address of IRQVECREG and FIQVECREG, which store the pending ISR address.

Example 15-4 shows a fast response to the FIQ interrupt in Index Interrupt and can be applied to a system that has more than one channel assigned as a FIQ. It is built in Index Interrupt compatible with TMS470R1x legacy code.

Example 15-4. How to Respond to FIQ With Short Latency

```
          .sect ".intvecs"          ; Interrupt and exception vector sector
00000000h b _RESET                  ; RESET interrupt
00000004h b _UNDEF_INST_INT         ; UNDEFINED INSTRUCTION interrupt
00000008h b _SW_INT                 ; SOFTWARE interrupt
0000000Ch b _ABORT_PREF_INT         ; ABORT (PREFETCH) interrupt
00000010h b _ABORT_DATA_INT         ; ABORT (DATA) interrupt
00000014h b #-8                     ; Reserved
00000018h b _IRQ_ENTRY_0            ; IRQ interrupt
                                    ;*********************************
                                    ; INTERRUPT PROCESSING AREA
                                    ;*********************************
0000001Ch ldrb R8, [PC,#-0x21d]     ; FIQ INTERRUPT ENTRY
                                    ; R8 used to get the FIQ index
                                    ; with address pointer to the
                                    ; first FIQ banked register
00000020h ldr PC, [PC, R8, LSL#2]   ; Branch to the indexed interrupt
                                    ; routine. The prefetch
                                    ; operation causes the PC to be 2
                                    ; words (8 bytes) ahead of the
                                    ; current instruction, so
                                    ; pointing to _INT_TABLE.
00000024h nop                       ; Required due to pipeline.
                                    ;================================
00000028h _INT_TABLE                ; FIQ INTERRUPT DISPATCH
                                    ;================================
0000002Ch .word _FIQ_TABLE          ; beginning of FIQ Dispatch
00000030h .word _ISR1               ; dispatch to interrupt routine 1
00000034h .word _ISR2               ; dispatch to interrupt routine 2
          .
          .
```

Another way to improve the FIQ latency is to assign only one channel to the FIQ interrupt and to map the ISR code corresponding to this channel directly starting at 0x1C.

NOTE: When the CPU is in vector-enabled mode, Example 15-3 and Example 15-4 are still valid. The difference is that the CPU will not read from the 0x18 location during IRQ interrupt, but will jump directly to the corresponding ISR routine.

15.8 VIM Registers

Table 15-1 lists the VIM module registers. Each register begins on a word boundary. All registers are 32-bit, 16-bit, and 8-bit accessible for read and write. Write is only possible in privilege mode. The base address of the control registers is FFFF FE00h. The base address of the parity-related VIM registers is FFFF FD00h. The address locations not listed are reserved.

Table 15-1. VIM Control Registers

Offset	Acronym	Register Description	Section
		Parity-related Registers	
ECh	PARFLG	Interrupt Vector Table Parity Flag Register	Section 15.8.1
F0h	PARCTL	Interrupt Vector Table Parity Control Register	Section 15.8.2
F4h	ADDERR	Address Parity Error Register	Section 15.8.3
F8h	FBPARERR	Fall-Back Address Parity Error Register	Section 15.8.4
		Control Registers	
00h	IRQINDEX	IRQ Index Offset Vector Register	Section 15.8.5.1
04h	FIQINDEX	FIQ Index Offset Vector Register	Section 15.8.5.2
10h	FIRQPR0	FIQ/IRQ Program Control Register 0	Section 15.8.6
14h	FIRQPR1	FIQ/IRQ Program Control Register 1	Section 15.8.6
18h	FIRQPR2	FIQ/IRQ Program Control Register 2	Section 15.8.6
1Ch	FIRQPR3	FIQ/IRQ Program Control Register 3	Section 15.8.6
20h	INTREQ0	Pending Interrupt Read Location Register 0	Section 15.8.7
24h	INTREQ1	Pending Interrupt Read Location Register 1	Section 15.8.7
28h	INTREQ2	Pending Interrupt Read Location Register 2	Section 15.8.7
2Ch	INTREQ3	Pending Interrupt Read Location Register 3	Section 15.8.7
30h	REQENASET0	Interrupt Enable Set Register 0	Section 15.8.8
34h	REQENASET1	Interrupt Enable Set Register 1	Section 15.8.8
38h	REQENASET2	Interrupt Enable Set Register 2	Section 15.8.8
3Ch	REQENASET3	Interrupt Enable Set Register 3	Section 15.8.8
40h	REQENACLR0	Interrupt Enable Clear Register 0	Section 15.8.9
44h	REQENACLR1	Interrupt Enable Clear Register 1	Section 15.8.9
48h	REQENACLR2	Interrupt Enable Clear Register 2	Section 15.8.9
4Ch	REQENACLR3	Interrupt Enable Clear Register 3	Section 15.8.9
50h	WAKEENASET0	Wake-up Enable Set Register 0	Section 15.8.10
54h	WAKEENASET1	Wake-up Enable Set Register 1	Section 15.8.10
58h	WAKEENASET2	Wake-up Enable Set Register 2	Section 15.8.10
5Ch	WAKEENASET3	Wake-up Enable Set Register 3	Section 15.8.10
60h	WAKEENACLR0	Wake-up Enable Clear Register 0	Section 15.8.11
64h	WAKEENACLR1	Wake-up Enable Clear Register 1	Section 15.8.11
68h	WAKEENACLR2	Wake-up Enable Clear Register 2	Section 15.8.11
6Ch	WAKEENACLR3	Wake-up Enable Clear Register 3	Section 15.8.11
70h	IRQVECREG	IRQ Interrupt Vector Register	Section 15.8.12
74h	FIQVECREG	FIQ Interrupt Vector Register	Section 15.8.13
78h	CAPEVT	Capture Event Register	Section 15.8.14
80h-FCh	CHANCTRL0 to CHANCTRL31	VIM Interrupt Control Register	Section 15.8.15

15.8.1 Interrupt Vector Table Parity Flag Register (PARFLG)

Figure 15-11 and Table 15-2 describe this register.

Figure 15-11. Interrupt Vector Table Parity Flag Register (PARFLG) [offset = ECh]

31	16
Reserved	
R-0	

15	1	0
Reserved		PARFLG
R-0		R/WP-0

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -*n* = value after reset

Table 15-2. Interrupt Vector Table Parity Flag Register (PARFLG) Field Descriptions

Bit	Field	Value	Description
31-1	Reserved	0	Reads return 0. Writes have no effect.
0	PARFLG		The PARFLG indicates that a parity error has been found and that the Interrupt Vector Table is bypassed. The resulting vector of any IRQ/FRQ interrupt is then the value contained in the FBPARERR register until this bit has been cleared.
		0	Read: No parity error has occurred. Write: No effect.
		1	Read: A parity error has occurred and the Interrupt Vector Table is bypassed. Write: The PARFLG is cleared and the interrupt vector can be read from the Interrupt Vector Table.

15.8.2 Interrupt Vector Table Parity Control Register (PARCTL)

Figure 15-12. Interrupt Vector Table Parity Control Register (PARCTL) [offset = F0h]

31	16
Reserved	
R-0	

15	9	8	7	4	3	0
Reserved		TEST	Reserved		PARENA	
R-0		R/WP-0	R-0		R/WP-5h	

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -*n* = value after reset

Table 15-3. Interrupt Vector Table Parity Control Register (PARCTL) Field Descriptions

Bit	Field	Value	Description
31-9	Reserved	0	Reads return 0. Writes have no effect.
8	TEST		This bit maps the parity bits into the Interrupt Vector Table frame to make them accessible by the CPU.
		0	Parity bits are not memory mapped.
		1	Parity bits are memory mapped.
7-4	Reserved	0	Reads return 0. Writes have no effect.
3-0	PARENA		VIM parity enable.
		5h	VIM parity is disabled.
		All other values	VIM parity is enabled.
			Note: To avoid soft error to disable VIM parity checking, it is recommended to write Ah to enable parity checking.

15.8.3 Address Parity Error Register (ADDERR)

The address parity error register gives the address of the first parity error location.

NOTE: No computation is needed when reading the complete register to retrieve the address in the Interrupt Vector Table.

This register will never be reset by a power-on reset nor any other reset source.

Figure 15-13. Address Parity Error Register (ADDERR) [offset = F4h]

31	16
Interrupt Vector Table offset	
R-FFF8h	

15	9	8	2	1	0
Interrupt Vector Table offset		ADDERR		Word offset	
R-0010 000b		R-x		R-0	

LEGEND: R = Read only; x = value is indeterminate; -*n* = value after reset

Table 15-4. Address Parity Error Register (PARERR) Field Descriptions

Bit	Field	Description
31-9	Interrupt Vector Table offset	Interrupt Vector Table offset. Reads are always FFF8 2xxxh; writes have no effect.
8-2	ADDERR	Address parity error register. This register gives the address of the first encountered parity error since the flag has been clear. Subsequent parity errors will not update this register until the PARFLG register has been cleared. **Note: This register is valid only when PARFLG is set (see Section 15.8.1).**
1-0	Word offset	Word offset. Reads are always 0; writes have no effect.

15.8.4 Fall-Back Address Parity Error Register (FBPARERR)

This register provides a fall-back address to the VIM if a parity error has occurred in the Interrupt Vector Table. Figure 15-14 and Table 15-5 describe this register.

NOTE: This register will never be reset by a power-on reset nor any other reset source.

Figure 15-14. Fall-Back Address Parity Error Register (FBPARERR) [offset = F8h]

31	16
FBPARERR	
R/WP-x	

15	0
FBPARERR	
R/WP-x	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; x = value is indeterminate; -*n* = value after reset

Table 15-5. Fall Back Address Parity Error Register (FBPARERR) Field Descriptions

Bit	Field	Value	Description
31-0	FBPARERR	0-FFFF FFFFh	Fall back address parity error. This register is used by the VIM if the Interrupt Vector Table has been corrupted. The contents of the IRQVECREG and FIQVECREG registers will reflect the value programmed in FBPARERR. The value provided to the VIC port will also reflect FBPARERR until the PARFLG register has been cleared. This register provides the address of the ISR that will restore the integrity of the Interrupt Vector Table.

15.8.5 VIM Offset Vector Registers

The VIM offset register provides the user with the numerical index value that represents the pending interrupt with the highest precedence. The register IRQINDEX holds the index to the highest priority IRQ interrupt; the register FIQINDEX holds the index to the highest priority FIQ interrupt. The index can be used to locate the interrupt routine in a dispatch table, as shown in Table 15-6.

Table 15-6. Interrupt Dispatch

IRQINDEX / FIQINDEX Register Bit Field	Highest Priority Pending Interrupt Enabled
0x00	No interrupt
0x01	Channel 0
:	:
0x7F	Channel 126
0x80	Channel 127

NOTE: Channel 127 has no dedicated interrupt vector table entry. Therefore, Channel 127 shall NOT be used in application.

The VIM offset registers are read only. They are updated continuously by the VIM. When an interrupt is serviced, the offset vectors show the index for the next highest pending interrupt or 0x0 if no interrupt is pending.

15.8.5.1 IRQ Index Offset Vector Register (IRQINDEX)

The IRQ offset register provides the user with the numerical index value that represents the pending IRQ interrupt with the highest priority. Figure 15-15 and Table 15-7 describe this register.

Figure 15-15. IRQ Index Offset Vector Register (IRQINDEX) [offset = 00h]

31	16
Reserved	
R-0	

15	8	7	0
Reserved		IRQINDEX	
R-0		R-0	

LEGEND: R = Read only; -*n* = value after reset

Table 15-7. IRQ Index Offset Vector Register (IRQINDEX) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	IRQINDEX	0-FFh	IRQ index vector. The least-significant bits represent the index of the IRQ pending interrupt with the highest precedence, as shown in Table 15-6. When no interrupts are pending, the least significant byte of IRQINDEX is 0. **Note:** A read of register IRQINDEX or IRQVECREG will cause IRQINDEX / IRQVECREG to reflect the index/ISR address for the next highest-priority pending IRQ interrupt. In case there is no other interrupt pending, the IRQINDEX will read 0x00 and the IRQVECREG register will read the phantom interrupt address.

15.8.5.2 FIQ Index Offset Vector Registers (FIQINDEX)

The FIQINDEX register provides the user with a numerical index value that represents the pending FIQ interrupt with the highest priority. Figure 15-16 and Table 15-8 describe this register.

Figure 15-16. FIQ Index Offset Vector Register (FIQINDEX) [offset = 04h]

31	16
Reserved	
R-0	

15	8	7	0
Reserved		FIQINDEX	
R-0		R-0	

LEGEND: R = Read only; -*n* = value after reset

Table 15-8. FIQ Index Offset Vector Register (FIQINDEX) Field Descriptions

Bit	Field	Value	Description
31-8	Reserved	0	Reads return 0. Writes have no effect.
7-0	FIQINDEX	0-FFh	FIQ index offset vector. The least-significant bits represent the index of the FIQ pending interrupt with the highest precedence, as shown in Table 15-6. When no interrupts are pending, the least significant byte of FIQINDEX is 0x00. **Note:** A read of register FIQINDEX or FIQVECREG will cause FIQINDEX / FIQVECREG to reflect the index/ISR address for the next highest-priority pending FIQ interrupt. In case there is no other interrupt pending, the FIQINDEX will read 0x00 and the FIQVECREG register will read the phantom interrupt address.

15.8.6 FIQ/IRQ Program Control Registers (FIRQPR[0:3])

The FIQ/IRQ program control registers (FIRQPR[0]) determine whether a given interrupt request will be either FIQ or IRQ. Figure 15-17, Figure 15-18, Figure 15-19, Figure 15-20 and Table 15-9 describe these registers.

NOTE: Channel 0 and 1 are FIQ only, not impacted by this register.

Figure 15-17. FIQ/IRQ Program Control Register 0 (FIRQPR0) [offset = 10h]

31	16
FIRQPR0[31:16]	
R/WP-0	

15	2	1	0
FIRQPR0[15:2]		Reserved	
R/WP-0		R-3h	

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-18. FIQ/IRQ Program Control Register 1 (FIRQPR1) [offset = 14h]

31	16
FIRQPR1[63:48]	
R/WP-0	

15	0
FIRQPR1[47:32]	
R/WP-0	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-19. FIQ/IRQ Program Control Register 2 (FIRQPR2) [offset = 18h]

31	16
FIRQPR2[95:80]	
R/WP-0	

15	0
FIRQPR2[79:64]	
R/WP-0	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-20. FIQ/IRQ Program Control Register 3 (FIRQPR3) [offset = 1Ch]

31	16
FIRQPR3[127:112]	
R/WP-0	

15	0
FIRQPR3[111:96]	
R/WP-0	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Table 15-9. FIQ/IRQ Program Control Registers (FIRQPR) Field Descriptions

Bit	Field	Value	Description
127-2	FIRQPRx[127:2]		FIQ/IRQ program control bits. These bits determine whether an interrupt request from a peripheral is of type FIQ or IRQ. Bit FIRQPRx[127:2] corresponds to request channel[127:2].
		0	Interrupt request is of IRQ type.
		1	Interrupt request is of FIQ type.
1-0	Reserved	3h	Read only. Writes have no effect.

15.8.7 Pending Interrupt Read Location Registers (INTREQ[0:3])

The pending interrupt register gives the pending interrupt requests. The register is updated every vbus clock cycle. Figure 15-21, Figure 15-22, Figure 15-23, Figure 15-24 and Table 15-10 describe this register.

Figure 15-21. Pending Interrupt Read Location Register 0 (INTREQ0) [offset = 20h]

31	16
INTREQ0[31:16]	
R/W1CP-0	
15	0
INTREQ0[15:0]	
R/W1CP-0	

LEGEND: R/W = Read/Write; W1CP = Write 1 to clear in privilege mode only; -*n* = value after reset

Figure 15-22. Pending Interrupt Read Location Register 1 (INTREQ1) [offset = 24h]

31	16
INTREQ1[63:48]	
R/W1CP-0	
15	0
INTREQ1[47:32]	
R/W1CP-0	

LEGEND: R/W = Read/Write; W1CP = Write 1 to clear in privilege mode only; -*n* = value after reset

Figure 15-23. Pending Interrupt Read Location Register 2 (INTREQ2) [offset = 28h]

31	16
INTREQ2[95:80]	
R/W1CP-0	
15	0
INTREQ2[79:64]	
R/W1CP-0	

LEGEND: R/W = Read/Write; W1CP = Write 1 to clear in privilege mode only; -*n* = value after reset

Figure 15-24. Pending Interrupt Read Location Register 3 (INTREQ3) [offset = 2Ch]

31	16
INTREQ3[127:112]	
R/W1CP-0	
15	0
INTREQ3[111:96]	
R/W1CP-0	

LEGEND: R/W = Read/Write; W1CP = Write 1 to clear in privilege mode only; -*n* = value after reset

Table 15-10. Pending Interrupt Read Location Registers (INTREQ) Field Descriptions

Bit	Field	Value	Description
127-0	INTREQx[127:0]		Pending interrupt bits. These bits determine whether an interrupt request is pending for the request channel between 0 and 127. The interrupt ENABLE register does not affect the value of the interrupt pending bit. Bit INTREQx[127:0] corresponds to request channel[127:0].
			User and Privilege Mode read:
		0	No interrupt event has occurred.
		1	An interrupt is pending.
			Privilege Mode write only:
		0	Writing 0 has no effect.
		1	Clears the "interrupt pending" status flag. This write-clear functionality is intended to allow clearing those interrupts which have been signaled to VIM before enabling the interrupt channel, if they are undesired.

15.8.8 Interrupt Enable Set Registers (REQENASET[0:3])

The interrupt register enable selectively enables individual request channels. Figure 15-25, Figure 15-26, Figure 15-27, Figure 15-28 and Table 15-11 describe these registers.

NOTE: Channel 0 and 1 are always enabled, not impacted by this register.

Figure 15-25. Interrupt Enable Set Register 0 (REQENASET0) [offset = 30h]

31	16
REQENASET0[31:16]	
R/WP-0	

15	2	1	0
REQENASET0[15:2]		Reserved	
R/WP-0		R-3h	

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-26. Interrupt Enable Set Register 1 (REQENASET1) [offset = 34h]

31	16
REQENASET1[63:48]	
R/WP-0	

15	0
REQENASET1[47:32]	
R/WP-0	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-27. Interrupt Enable Set Register 2 (REQENASET2) [offset = 38h]

31	16
REQENASET295:80]	
R/WP-0	

15	0
REQENASET2[79:64]	
R/WP-0	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-28. Interrupt Enable Set Register 3 (REQENASET3) [offset = 3Ch]

31	16
REQENASET3127:112]	
R/WP-0	

15	0
REQENASET3[111:96]	
R/WP-0	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Table 15-11. Interrupt Enable Set Registers (REQENASET) Field Descriptions

Bit	Field	Value	Description
127-2	REQENASETx[127:2]		Request enable set bits. This vector determines whether the interrupt request channel is enabled. Bit REQENASETx[127:2] corresponds to request channel[127:2].
		0	Read: Interrupt request channel is disabled. Write: No effect.
		1	Read or Write: The interrupt request channel is enabled.
1-0	Reserved	3h	Read only. Writes have no effect.

15.8.9 Interrupt Enable Clear Registers (REQENACLR[0:3])

The interrupt register enable selectively disables individual request channels. Figure 15-29, Figure 15-30, Figure 15-31, Figure 15-32 and Table 15-12 describe these registers.

NOTE: Channel 0 and 1 are always enabled, not impacted by this register.

Figure 15-29. Interrupt Enable Clear Register 0 (REQENACLR0) [offset = 40h]

31	16
REQENACLR0[31:16]	
R/WP-0	

15	2	1	0
REQENACLR0[15:2]		Reserved	
R/WP-0		R-3h	

LEGEND: R/W = Read/Write; R = Read only; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-30. Interrupt Enable Clear Register 1 (REQENACLR1) [offset = 44h]

31	16
REQENACLR1[63:48]	
R/WP-0	

15	0
REQENACLR1[47:32]	
R/WP-0	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-31. Interrupt Enable Clear Register 2 (REQENACLR2) [offset = 48h]

31	16
REQENACLR2[95:80]	
R/WP-0	

15	0
REQENACLR2[79:64]	
R/WP-0	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-32. Interrupt Enable Clear Register 3 (REQENACLR3) [offset = 4Ch]

31	16
REQENACLR3[127:112]	
R/WP-0	

15	0
REQENACLR3[111:96]	
R/WP-0	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

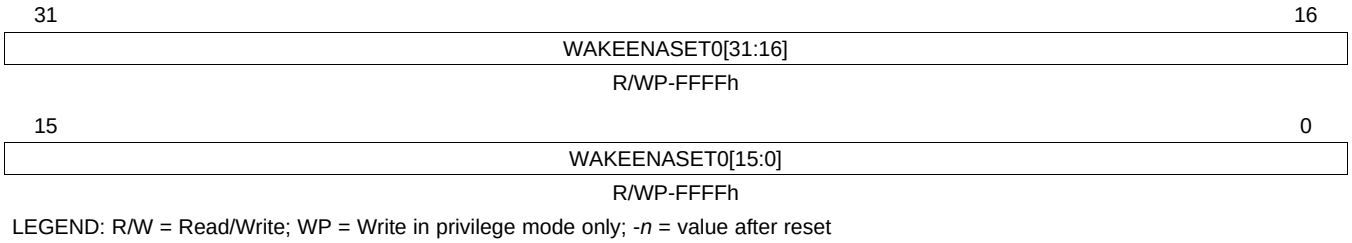
Table 15-12. Interrupt Enable Clear Registers (REQENACLR) Field Descriptions

Bit	Field	Value	Description
127-2	REQENACLRx[127:2]		Request enable clear bits. This vector determines whether the interrupt request channel is enabled. Bit REQENACLRx[127:2] corresponds to request channel[127:2].
		0	Read: Interrupt request channel is disabled. Write: No effect.
		1	Read: The interrupt request channel is enabled. Write: The interrupt request channel is disabled.
1-0	Reserved	3h	Read only. Writes have no effect.

15.8.10 Wake-Up Enable Set Registers (WAKEENASET[0:3])

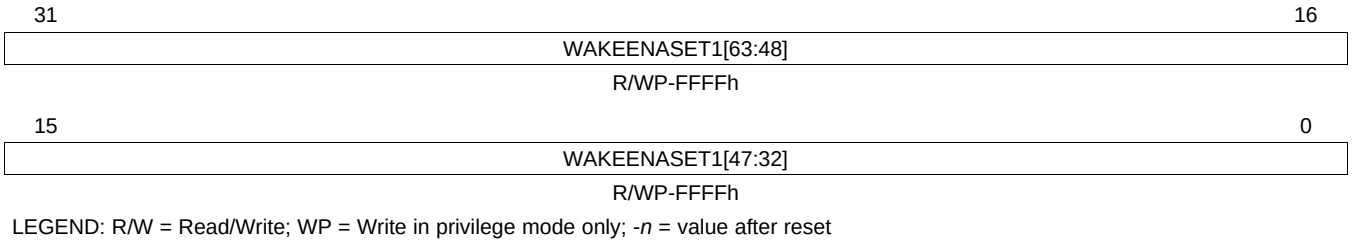
The wake-up enable registers selectively enables individual wake-up interrupt request lines. Figure 15-33, Figure 15-34, Figure 15-35, Figure 15-36 and Table 15-13 describe these registers.

Figure 15-33. Wake-Up Enable Set Register 0 (WAKEENASET0) [offset = 50h]



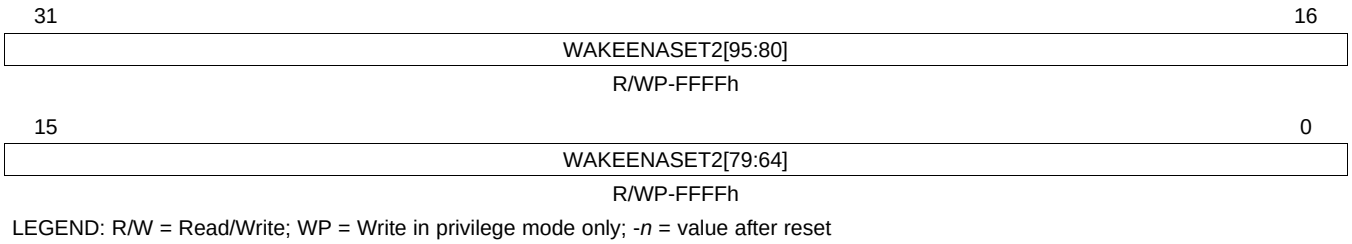
LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-34. Wake-Up Enable Set Register 1 (WAKEENASET1) [offset = 54h]



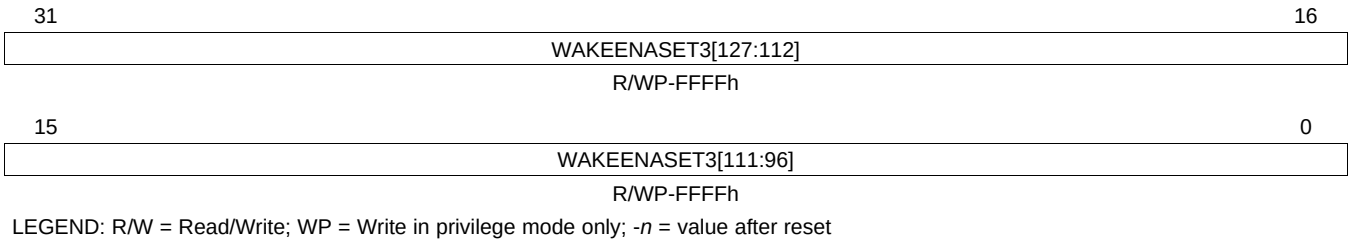
LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-35. Wake-Up Enable Set Register 2 (WAKEENASET2) [offset = 58h]



LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-36. Wake-Up Enable Set Register 3 (WAKEENASET3) [offset = 5Ch]



LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Table 15-13. Wake-Up Enable Set Registers (WAKEENASET) Field Descriptions

Bit	Field	Value	Description
127-0	WAKEENASETx[127:0]		Wake-up enable set bits. This vector determines whether the wake-up interrupt line is enabled. Bit WAKEENASETx[127:0] corresponds to interrupt request channel[127:0].
		0	Read: Interrupt request channel is disabled. Write: No effect.
		1	Read or Write: The interrupt request channel is enabled.

15.8.11 Wake-Up Enable Clear Registers (WAKEENACLR[0:3])

The wake-up enable register selectively disables individual wake-up interrupt request lines. Figure 15-37, Figure 15-38, Figure 15-39, Figure 15-40 and Table 15-14 describe these registers.

Figure 15-37. Wake-Up Enable Clear Register 0 (WAKEENACLR0) [offset = 60h]

31	16
WAKEENACLR0[31:16]	
R/WP-FFFFh	
15	0
WAKEENACLR0[15:0]	
R/WP-FFFFh	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-38. Wake-Up Enable Clear Register 1 (WAKEENACLR1) [offset = 64h]

31	16
WAKEENACLR1[63:48]	
R/WP-FFFFh	
15	0
WAKEENACLR1[47:32]	
R/WP-FFFFh	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-39. Wake-Up Enable Clear Register 2 (WAKEENACLR2) [offset = 68h]

31	16
WAKEENACLR2[95:80]	
R/WP-FFFFh	
15	0
WAKEENACLR2[79:64]	
R/WP-FFFFh	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Figure 15-40. Wake-Up Enable Clear Register 3 (WAKEENACLR3) [offset = 6Ch]

31	16
WAKEENACLR3[127:112]	
R/WP-FFFFh	
15	0
WAKEENACLR3[111:96]	
R/WP-FFFFh	

LEGEND: R/W = Read/Write; WP = Write in privilege mode only; -*n* = value after reset

Table 15-14. Wake-Up Enable Clear Registers (WAKEENACLR) Field Descriptions

Bit	Field	Value	Description
127-0	WAKEENACLRx[127:0]		Wake-up enable clear bits. This vector determines whether the wake-up interrupt line is enabled. Bit WAKEENACLRx[127:0] corresponds to interrupt request channel[127:0].
		0	Read: Wake-up interrupt channel is disabled. Write: No effect.
		1	Read: The wake-up interrupt channel is enabled. Write: The wake-up interrupt channel is disabled.

15.8.12 IRQ Interrupt Vector Register (IRQVECREG)

The interrupt vector register gives the address of the enabled and active IRQ interrupt. Figure 15-41 and Table 15-15 describe these registers.

Figure 15-41. IRQ Interrupt Vector Register (IRQVECREG) [offset = 70h]

31	0
IRQVECREG	
R-0	

LEGEND: R = Read only; -*n* = value after reset

Table 15-15. IRQ Interrupt Vector Register (IRQVECREG) Field Descriptions

Bit	Field	Value	Description
31-0	IRQVECREG	From Section 15.4	IRQ interrupt vector register. This vector gives the address of the ISR with the highest pending IRQ request. The CPU reads the address and branches to this location.

15.8.13 FIQ Interrupt Vector Register (FIQVECREG)

The interrupt vector register gives the address of the enabled and active FIQ interrupt. Figure 15-42 and Table 15-16 describe these registers.

Figure 15-42. IRQ Interrupt Vector Register (FIQVECREG) [offset = 74h]

31	0
FIQVECREG	
R-0	

LEGEND: R = Read only; -*n* = value after reset

Table 15-16. FIQ Interrupt Vector Register (FIQVECREG) Field Descriptions

Bit	Field	Value	Description
31-0	FIQVECREG	From Section 15.4	FIQ interrupt vector register. This vector gives the address of the ISR with the highest pending FIQ request. The CPU reads the address and branches to this location.

15.8.14 Capture Event Register (CAPEVT)

Figure 15-43 and Table 15-17 describe this register.

Figure 15-43. Capture Event Register (CAPEVT) [offset = 78h]

31	23	22	16
Reserved		CAPEVTSRC1	
R-U		R/W-0	

15	7	6	0
Reserved		CAPEVTSRC0	
R-U		R/W-0	

LEGEND: R/W = Read/Write; R = Read only; U = value is undefined; -*n* = value after reset

Table 15-17. Capture Event Register (CAPEVT) Field Descriptions

Bit	Field	Value	Description
31-23	Reserved	0	Reads are indeterminate and writes have no effect.
22-16	CAPEVTSRC1		Capture event source 1 mapping control. These bits determine which interrupt request maps to the capture event source 1 of the RTI:
		0	Interrupt request 0.
		1h	Interrupt request 1.
		:	:
		7Fh	Interrupt request 127.
15-7	Reserved	0	Reads are indeterminate and writes have no effect.
6-0	CAPEVTSRC0		Capture event source 0 mapping control. These bits determine which interrupt request maps to the capture event source 0 of the RTI:
		0	Interrupt request 0.
		1h	Interrupt request 1.
		:	:
		7Fh	Interrupt request 127.

15.8.15 VIM Interrupt Control Registers (CHANCTRL[0:31])

Thirty-two interrupt control registers control the 128 interrupt channels of the VIM. Each register controls four interrupt channels: each of them is indexed from 0 to 127. Table 15-18 shows the organization of all the registers and the reset value of each. Each four fields of the register has been named with a generic index that refers to the detailed register organization. Figure 15-44 and Table 15-19 describe these registers.

Table 15-18. Interrupt Control Registers Organization

Address	Register Acronym	Register Field 31:24 $CHANMAPx_0$	Register Field 23:16 $CHANMAPx_1$	Register Field 15:8 $CHANMAPx_2$	Register Field 7:0 $CHANMAPx_3$	Reset Value
FFFF FE80h	CHANCTRL0	CHANMAP0	CHANMAP1	CHANMAP2	CHANMAP3	0001 0203h
FFFF FE84h	CHANCTRL1	CHANMAP4	CHANMAP5	CHANMAP6	CHANMAP7	0405 0607h
:	:	:	:	:	:	:
FFFF FEF8h	CHANCTRL30	CHANMAP120	CHANMAP121	CHANMAP122	CHANMAP123	7879 7A7Bh
FFFF FEFCh	CHANCTRL31	CHANMAP124	CHANMAP125	CHANMAP126	CHANMAP127	7C7D 7E7Fh

NOTE: CHANMAP0 and CHANMAP1 are not programable. CHAN0 and CHAN1 are hard wired to INT_REQ0 and INT_REQ1.

Do NOT write any value other than 0x7F to CHANMAP127. Channel 127 is reserved because no interrupt vector table entry supports this channel.

Figure 15-44. Interrupt Control Registers (CHANCTRL[0:31]) [offset = 80h-FCh]

31	30 24	23	22 16
Rsvd	$CHANMAPx_0$	Rsvd	$CHANMAPx_1$
R-U	R/W-n	R-U	R/W-n

15	14 8	7	6 0
Rsvd	$CHANMAPx_2$	Rsvd	$CHANMAPx_3$
R-U	R/W-n	R-U	R/W-n

LEGEND: R/W = Read/Write; R = Read only; U = value is undefined; -*n* = value after reset (see Table 15-18)

Table 15-19. Interrupt Control Registers (CHANCTL) Field Descriptions

Bit	Field	Value	Description
31	Reserved	0	Reads are indeterminate and writes have no effect.
30-24	$CHANMAPx_0$		$CHANMAPx_0$(6-0). Interrupt $CHANx_0$ mapping control. These bits determine which interrupt request the priority channel $CHANx_0$ maps to:
		0	Read: Interrupt request 0 maps to channel priority $CHANx_0$. Write: The default value of this bit after reset is given in Table 15-18 . The channel priority $CHANx_0$ is set with the interrupt request.
		1h	Read: Interrupt request 1 maps to channel priority $CHANx_0$. Write: The default value of this bit after reset is given in Table 15-18. The channel priority $CHANx_0$ is set with the interrupt request.
		:	:
		7Fh	Read: Interrupt request 127 maps to channel priority $CHANx_0$. Write: The default value of this bit after reset is given in Table 15-18. The channel priority $CHANx_0$ is set with the interrupt request.
23	Reserved	0	Reads are indeterminate and writes have no effect.

Table 15-19. Interrupt Control Registers (CHANCTL) Field Descriptions (continued)

Bit	Field	Value	Description
22-16	$CHANMAPx_1$		$CHANMAPx_1$(6-0). Interrupt $CHANx_1$ mapping control. These bits determine which interrupt request the priority channel $CHANx_1$ maps to:
		0	Read: Interrupt request 0 maps to channel priority $CHANx_1$. Write: The default value of this bit after reset is given in Table 15-18. The channel priority $CHANx_1$ is set with the interrupt request.
		1h	Read: Interrupt request 1 maps to channel priority $CHANx_1$. Write: The default value of this bit after reset is given in Table 15-18. The channel priority $CHANx_1$ is set with the interrupt request.
		:	:
		7Fh	Read: Interrupt request 127 maps to channel priority $CHANx_1$. Write: The default value of this bit after reset is given in Table 15-18. The channel priority $CHANx_1$ is set with the interrupt request.
15	Reserved	0	Reads are indeterminate and writes have no effect.
14-8	$CHANMAPx_2$		$CHANMAPx_2$(6-0). Interrupt $CHANx_2$ mapping control. These bits determine which interrupt request the priority channel $CHANx_2$ maps to:
		0	Read: Interrupt request 0 maps to channel priority $CHANx_2$. Write: The default value of this bit after reset is given in Table 15-18. The channel priority $CHANx_2$ is set with the interrupt request.
		1h	Read: Interrupt request 1 maps to channel priority $CHANx_2$. Write: The default value of this bit after reset is given in Table 15-18. The channel priority $CHANx_2$ is set with the interrupt request.
		:	:
		7Fh	Read: Interrupt request 127 maps to channel priority $CHANx_2$. Write: The default value of this bit after reset is given in Table 15-18. The channel priority $CHANx_2$ is set with the interrupt request.
7	Reserved	0	Reads are indeterminate and writes have no effect.
6-0	$CHANMAPx_3$		$CHANMAPx_3$(6-0). Interrupt $CHANx_3$ mapping control. These bits determine which interrupt request the priority channel $CHANx_3$ maps to:
		0	Read: Interrupt request 0 maps to channel priority $CHANx_3$. Write: The default value of this bit after reset is given in Table 15-18. The channel priority $CHANx_3$ is set with the interrupt request.
		1h	Read: Interrupt request 1 maps to channel priority $CHANx_3$. Write: The default value of this bit after reset is given in Table 15-18. The channel priority $CHANx_3$ is set with the interrupt request.
		:	:
		7Fh	Read: Interrupt request 127 maps to channel priority $CHANx_3$. Write: The default value of this bit after reset is given in Table 15-18. The channel priority $CHANx_3$ is set with the interrupt request.